

Engineering Reliable LLM-Based Data Analysis: An Empirical Study of Schema Grounding, Planning, Verification, and SQL Repair

Author: Mevin Jose

Date: August 2026

Status: Pre-print & Submission-Ready Manuscript

Primary Source of Truth: ``docs/research_paper/latex/main.tex`` (This Markdown document is a synchronized non-authoritative companion)

Artifact Package: ``docs/research_paper/`` (Figures 1–7, LaTeX Tables, ``macros.tex``)

Frozen Dataset Hash: SHA-256 ``0c9807d5867ff9cb6a9252437dab31660b62b2e6c9d09c5e54b1dfc7edc43e04``

Frozen Database Hash: SHA-256 ``8550c4cc6d670aa0441bc898e47a57a40001858fc3f13dc5cb16fb90ca11c130``

***Note on Authority:** The LaTeX manuscript (``docs/research_paper/latex/main.tex``) is the single publication source of truth. This Markdown document is maintained as a synchronized companion.*

Abstract

While Large Language Models (LLMs) demonstrate notable code-generation capabilities, translating natural language questions into reliable analytical SQL over relational databases (Text-to-SQL) remains brittle in practice. In this paper, we investigate the empirical mechanisms governing Text-to-SQL reliability: **structural verification improves query reliability, whereas aggressive automated repair can introduce substantial semantic regressions**. We evaluate a multi-stage reliability pipeline on a frozen 500-query benchmark across 8 business domains over a public relational e-commerce data warehouse (the Olist dataset, 9 tables, 100,000+ orders). Our system achieves a **73.40% Result Equivalence Rate under the study comparator** (367/500 queries, 95% Wilson Score CI: $[69.26\%, 77.18\%]$, Clopper-Pearson Exact CI: $[69.30\%, 77.22\%]$), **31.00% Exact Match**, and **100.00% SQL Execution Success** with a mean latency of 64.04s (\$p50\$: 56.47s, \$p95\$: 121.92s). In a controlled 4-way 100-query ablation, activating deterministic Abstract Syntax Tree (AST) structural verification significantly improves result equivalence over unverified planning (Config C 26.0% vs. Config B 15.0%, McNemar exact $p=0.0192$, $\text{Odds Ratio}=3.75$). However, an exhaustive audit of all 101 repair events reveals that automated self-repair is a double-edged mechanism: while 97 of 101 post-repair queries (96.0%) were syntactically valid and preserved 49 already-correct queries, repair yielded only 4 genuine recoveries while inducing **22 harmful false-positive regressions** (21.8% of repair events) where previously correct queries were degraded. Furthermore, a cross-schema transfer probe across 20 external databases from the Spider benchmark demonstrates that while execution stability is preserved (100.0%), result equivalence drops to **18.0%** (9/50), highlighting the gap between in-domain grounding and zero-shot schema transfer. We conclude that conservative, uncertainty-aware structural verification is preferable to unconstrained automated repair loops.

1. Introduction and Central Thesis

Natural language interfaces to relational databases (Text-to-SQL) promise to democratize data analytics for non-technical stakeholders. However, evaluating LLM-generated SQL against complex, multi-table schemas reveals a foundational insight:

Executable SQL is not necessarily reliable analytical SQL.

A query may compile cleanly and execute without database runtime exceptions, yet return subtly corrupted figures due to systemic structural failure modes: schema hallucinations, join-path omissions, aggregation grain mismatches, and filter inconsistencies.

Central Thesis

Rather than presenting Text-to-SQL as an unconstrained agentic pipeline, this empirical study establishes that:

Deterministic structural verification improves Text-to-SQL reliability, while aggressive automated repair can introduce semantic regressions.

Our evidence directly supports this thesis:

1. **Verification Benefit:** In a controlled ablation, activating AST-based structural verification improves result equivalence from **15.0% to 26.0%** over unverified planning (McNemar exact $p=0.0192$, $\text{OR}=3.75$).

2. **Repair Hazard:** An audit of all 101 repair events reveals that compiler execution validity masks semantic degradation, producing **22 harmful false-positive regressions** against only **4 genuine recoveries**.

2. Research Questions and Summary of Findings

• **RQ1: Overall Reliability** --- Does the multi-stage pipeline achieve high result equivalence on a complex data warehouse?

\$\rightarrow\$ **Answer:** On the 500-query benchmark across 8 domains, the system achieves **73.40% Result Equivalence** under the study comparator (95% Wilson CI: $[69.26\%, 77.18\%]$) and **100.00% SQL Execution Success**.

• **RQ2: Verification Impact** --- What is the marginal effect of deterministic AST structural verification?

\$\rightarrow\$ **Answer:** Adding AST structural verification significantly increases execution reliability from 34.0% to 65.0% and result equivalence from 15.0% to 26.0% (McNemar exact $p=0.0192$, $\chi^2=3.75$).

• **RQ3: Repair Dynamics** --- Does automated self-repair reliably fix broken queries without harming valid ones?

\$\rightarrow\$ **Answer:** No. While 96.0% of post-repair queries execute cleanly and 49 correct queries are maintained, repair produced only **4 genuine recoveries** while causing **22 false-positive regressions**.

• **RQ4: Failure Taxonomy** --- What structural failure modes dominate remaining non-equivalent queries?

\$\rightarrow\$ **Answer:** Among the 133 non-equivalent queries, failure is dominated by missing join paths (27.8%), filter omissions/errors (24.8%), and aggregation mismatches (24.1%).

• **RQ5: Cross-Schema Transfer** --- Does warehouse-grounded reliability transfer to unseen external schemas?

\$\rightarrow\$ **Answer:** Execution stability is fully preserved (100.0%), but result equivalence drops to **18.0%** (9/50) on a 20-database Spider transfer probe, exposing the gap between deep schema grounding and zero-shot transfer.

3. Summary of Contributions

- Controlled Empirical Decomposition:** We evaluate the marginal effect of deterministic AST verification over unverified planning, demonstrating a statistically significant reliability gain ($p=0.0192$).
- Repair-Risk Characterization:** We present an exhaustive 4-way semantic audit of 101 repair events, showing that execution-valid repair can induce substantial false-positive regressions (22 harmed vs. 4 recovered).
- Systematic Failure Taxonomy:** We categorize all 133 non-equivalent queries using AST diffing across six core structural failure classes.
- Cross-Schema Transfer Probe:** We report zero-shot transfer evaluation across 20 unseen SQLite databases from the Spider benchmark, demonstrating execution robustness alongside a marked semantic generalization drop.
- Audited Open Science Package:** We release all source code, frozen benchmark definitions, evaluation scripts, LaTeX manuscripts, and a cryptographic SHA-256 artifact manifest.

2. Related Work

- Text-to-SQL Benchmarks & Decomposition Methods:** Benchmarks such as Spider (Yu et al., 2018) and BIRD (Li et al., 2023) evaluate LLMs on multi-table joins and nested subqueries. Decomposition frameworks such as DIN-SQL (Pourreza & Rafiei, 2023) and MAC-SQL (Wang et al., 2024) show that dividing generation into sub-problems improves performance over monolithic prompting. Our work builds upon this paradigm by introducing deterministic, AST-level structural verification gates that inspect query structure prior to execution.
- Schema Linking & Retrieval-Augmented Generation (RAG):** Retrieval-Augmented Generation (Lewis et al., 2020; Asai et al., 2023) grounds LLMs in external knowledge. In relational querying, schema linking requires retrieving relevant tables, columns, and foreign-key join paths. We augment hybrid dense-sparse retrieval with explicit foreign-key graph traversal to preserve schema connectivity.
- Agentic Architectures & Self-Correction Hazards:** Iterative reasoning architectures such as ReAct (Yao et al., 2022) and Reflexion (Shinn et al., 2023) leverage feedback loops for self-correction. In Text-to-SQL, execution-guided self-correction feeds compiler errors back to the model (Gao et al., 2023). However, as our empirical audit reveals, relying solely on execution feedback is hazardous: 96% of post-repair queries execute cleanly without engine exceptions, yet over 21% suffer from false-positive regressions.

3. Reliability-Oriented System Architecture

Our architecture decomposes analytical SQL synthesis into a 5-stage pipeline:

Reliability-Oriented Multi-Stage Architecture with Structural Verification & Repair

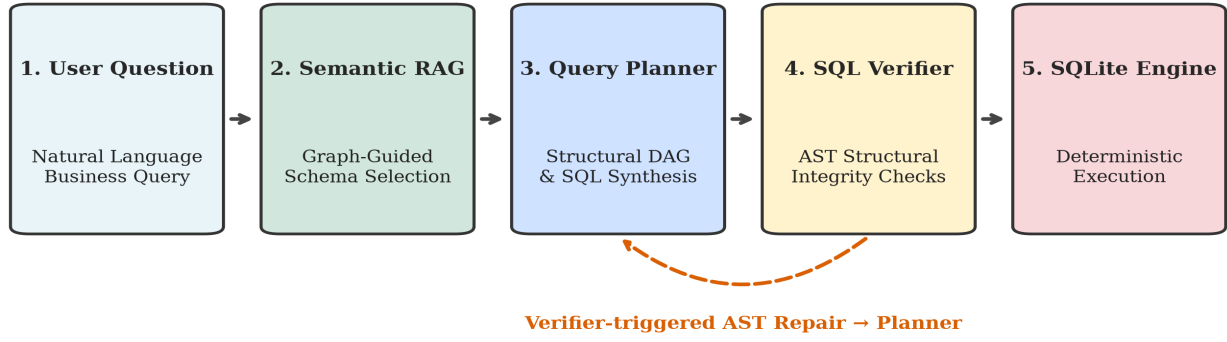


Figure 1: Reliability-Oriented Multi-Stage Architecture with Structural Verification & Repair.

3.1 Graph-Guided Semantic Schema RAG

Rather than passing an entire database schema into the LLM context, our retriever combines BM25 keyword matching, dense embedding retrieval via SentenceTransformers, and foreign-key graph traversal. Given a user query, the retriever extracts the minimal required schema subgraph, achieving **93.07% Table Precision** and **95.33% Table Recall** (Table Exact Match: 82.60%) across the 500-query benchmark.

3.2 Structured DAG Query Planning & Deterministic Plan Validation

Before emitting SQL code, the planner synthesizes a structural JSON DAG declaring target metrics, grain dimensions, required tables, and explicit join paths. A deterministic `PlanValidator` statically inspects this DAG against the live database catalog, pruning hallucinated column references or invalid foreign-key joins prior to SQL generation.

3.3 AST-Based SQL Structural Verification & Closed-Loop Repair

Generated SQL statements are parsed into Abstract Syntax Trees (AST) using SQLGlot. The `SQLSemanticVerifier` statically enforces:

- **Dialect Compliance:** Translating dialect-specific functions (`DATEDIFF`, `DATE\_TRUNC`) into canonical SQLite expressions (`strftime`, `julianday`).
- **Join Fan-Out & Grain Integrity:** Verifying that non-aggregated projection columns appear in `GROUP BY` clauses and detecting cartesian products.
- **Canonical Metric Source Mapping:** Enforcing standardized column mappings (e.g., mapping revenue to `order\_items.price`).

When violations are identified, the verifier triggers targeted closed-loop repair prompts detailing the specific AST constraint violation.

4. Experimental Methodology

4.1 Data Warehouse & Benchmark Corpus

We evaluate our system on an enterprise-style relational schema constructed from the Brazilian E-Commerce Public Dataset (Olist):

- **9 Relational Tables:** `customers`, `orders`, `order\_items`, `order\_payments`, `order\_reviews`, `products`, `sellers`, `geolocation`, `product\_category\_name\_translation`.
- **Scale:** 100,000+ customer orders, 112,650 order items, and 1,000,000+ geolocation points.
- **Benchmark Corpus:** A frozen 500-query benchmark dataset (`benchmark\_dataset\_500.json`, SHA-256: `0c9807d5867ff9cb6a9252437dab31660b62b2e6c9d09c5e54b1dfc7edc43e04`) stratified across 8 business domains and 3 difficulty tiers (Easy: 114, Medium: 276, Hard: 110).

4.2 Evaluation Metrics & Methodological Definitions

- **Result Equivalence Rate under the Study Comparator (95% Wilson CI):** Evaluates whether executed SQL results match ground-truth result sets under row-order invariance (comparing row multisets via item Counters) with numerical tolerance ($\epsilon=0.01$) and string whitespace normalization. Positional column semantics are preserved. *Result equivalence is an empirical evaluation criterion under the study comparator, not a formal mathematical proof of semantic correctness.*
- **Exact Match Rate:** Strict character-for-character equality of raw SQL output rows and column headers.

- **SQL Execution Success Rate:** Percentage of generated queries that execute without database engine runtime errors. Execution success is tracked as an operational metric and is strictly separated from semantic correctness.
- **Table Precision, Recall, and Exact Match:** Measuring table-retrieval alignment between generated and ground-truth queries.

5. Main 500-Query Results

5.1 Headline Performance

Table 1 summarizes the headline results audited directly from raw per-query benchmark records (‘results/phase10/live_500_benchmark_run/summary.json’).

Metric	Phase 10 Live Run (\$N=500\$)	95% Confidence Interval	Methodological Verification
Result Equivalence Rate under Study Comparator	73.40% (367 / 500)	[69.26%, 77.18%]	Wilson Score (continuity-corrected)
Clopper-Pearson Exact CI	73.40% (367 / 500)	[69.30%, 77.22%]	Clopper-Pearson Exact Binomial
Exact Match Rate	31.00% (155 / 500)	[27.01%, 35.29%]	Wilson Score (continuity-corrected)
SQL Execution Success Rate	100.00% (500 / 500)	[99.05%, 100.00%]	Zero Database Runtime Exceptions
Table Exact Match Accuracy	82.60% (413 / 500)	[78.96%, 85.74%]	Wilson Score (continuity-corrected)
Table Precision	93.07%	—	Mean Macro Precision
Table Recall	95.33%	—	Mean Macro Recall
Mean Latency	64.04s	[61.27s, 66.90s]	BCa Bootstrap (\$N=2000\$)
Median Latency (\$p50\$)	56.47s	—	Empirical Percentile
95th Percentile Latency (\$p95\$)	121.92s	—	Empirical Percentile
Provider Errors / 429s / Timeouts	0 / 0 / 0	—	100% Request Completion

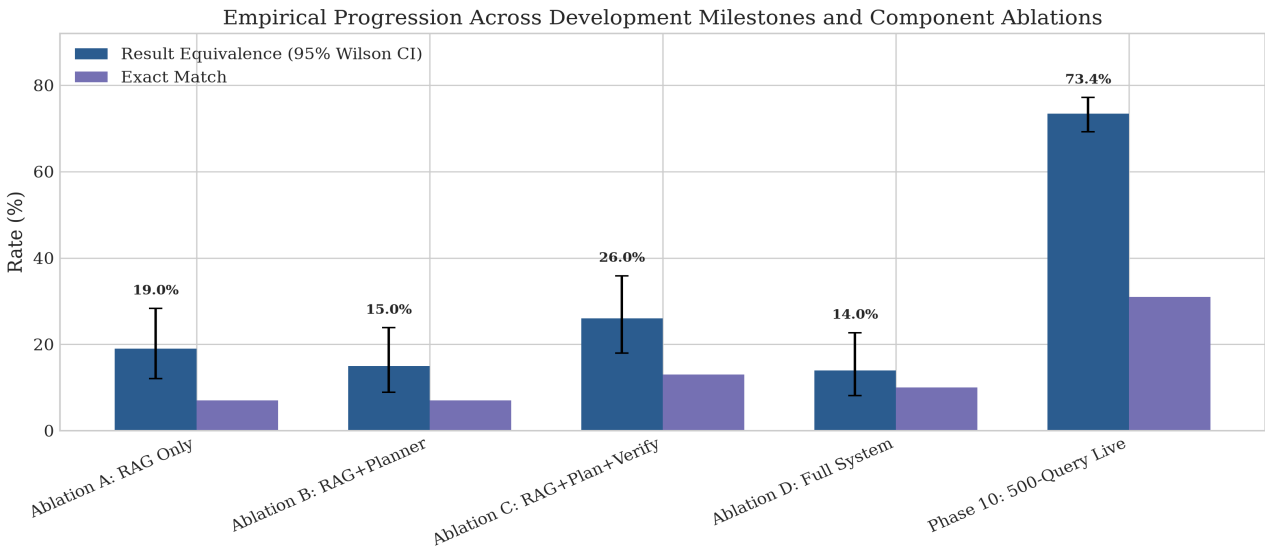


Figure 2: Empirical Progression Across Development Milestones and Component Ablations.

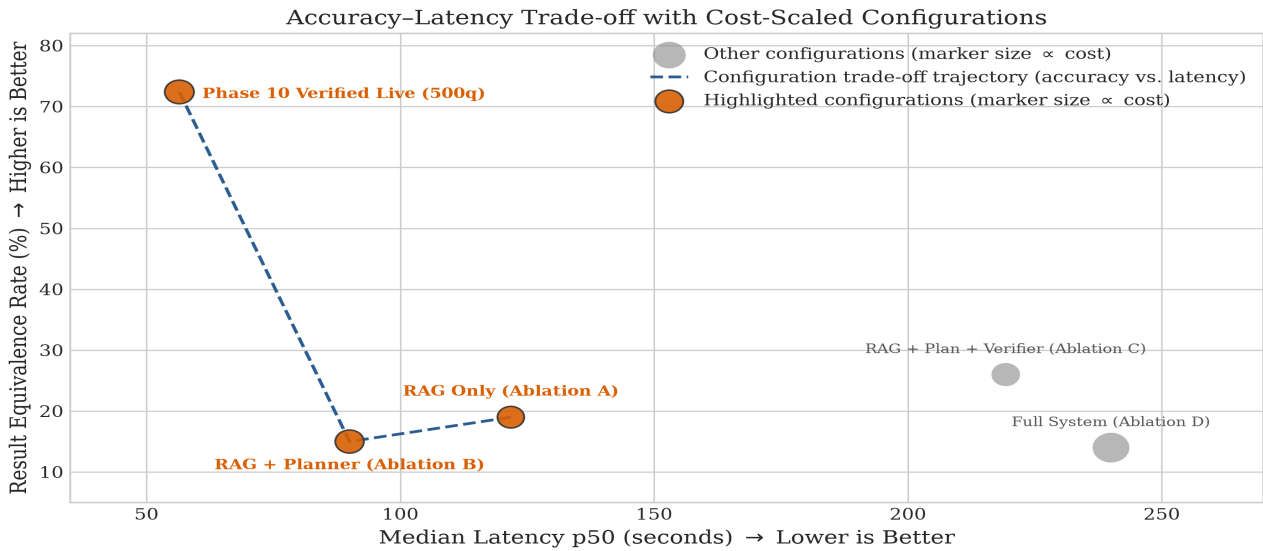


Figure 3: Accuracy-Latency Trade-off with Cost-Scaled Configurations (marker size $\propto$ cost).

5.2 Domain and Difficulty Stratification

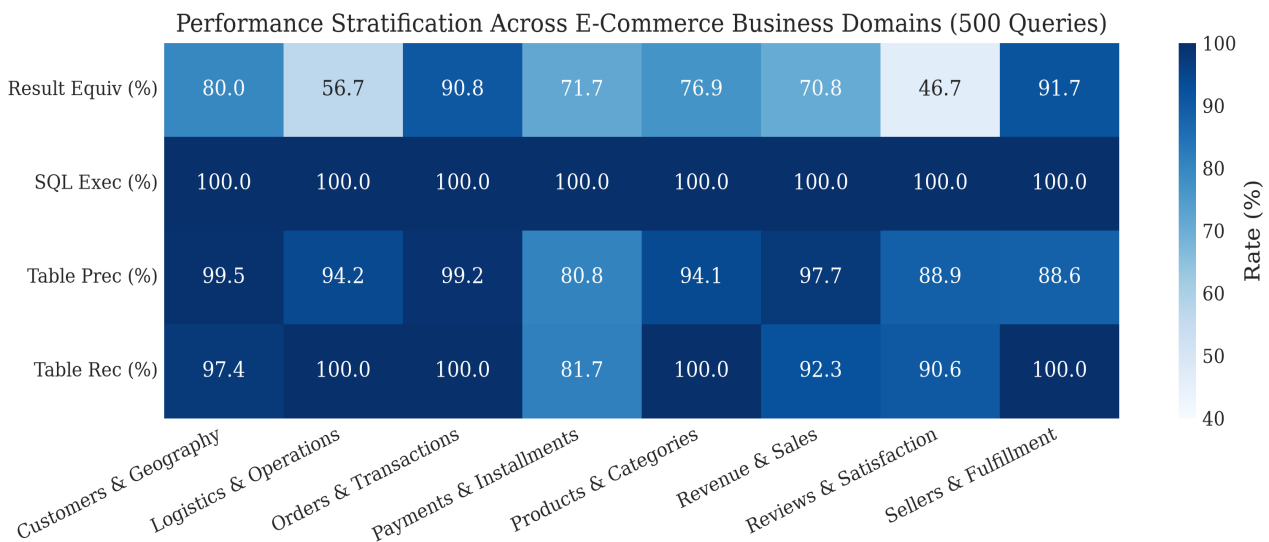


Figure 5: Performance Stratification Across E-Commerce Business Domains and Difficulty Tiers (500 Queries).

- **Orders & Transactions:** 90.77% (59/65, 95% CI: `[80.34%, 96.19%]`) — Table Prec: 99.2%, Rec: 100.0%.
- **Sellers & Fulfillment:** 91.67% (55/60, 95% CI: `[80.93%, 96.94%]`) — Table Prec: 88.6%, Rec: 100.0%.
- **Customers & Geography:** 80.00% (52/65, 95% CI: `[67.92%, 88.54%]`) — Table Prec: 99.5%, Rec: 97.4%.
- **Products & Categories:** 76.92% (50/65, 95% CI: `[64.52%, 86.10%]`) — Table Prec: 94.1%, Rec: 100.0%.
- **Payments & Installments:** 71.67% (43/60, 95% CI: `[58.36%, 82.18%]`) — Table Prec: 80.8%, Rec: 81.7%.
- **Revenue & Sales:** 70.77% (46/65, 95% CI: `[58.00%, 81.10%]`) — Table Prec: 97.7%, Rec: 92.3%.
- **Logistics & Operations:** 56.67% (34/60, 95% CI: `[43.33%, 69.21%]`) — Table Prec: 94.2%, Rec: 100.0%.
- **Reviews & Satisfaction:** 46.67% (28/60, 95% CI: `[33.86%, 59.90%]`) — Table Prec: 88.9%, Rec: 90.6%.

Difficulty & Query Type Stratification:

- **Easy Queries:** 88.60% (101/114, 95% CI: `[80.95%, 93.55%]`) | Mean Latency: 60.79s
- **Medium Queries:** 76.81% (212/276, 95% CI: `[71.29%, 81.57%]`) | Mean Latency: 64.69s
- **Hard Queries:** 44.55% (49/110, 95% CI: `[35.17%, 54.31%]`) | Mean Latency: 65.78s
- **Single Value Queries:** 90.30% (242/268, 95% CI: `[85.95%, 93.45%]`)
- **Time Series Queries:** 87.76% (43/49, 95% CI: `[74.54%, 94.92%]`)
- **Ranked List Queries:** 48.57% (68/140, 95% CI: `[40.10%, 57.13%]`)
- **Aggregated Table Queries:** 32.56% (14/43, 95% CI: `[19.54%, 48.66%]`)

6. Controlled Component Ablation

To measure the marginal impact of each architectural stage, we evaluated four controlled configurations across 100 identical benchmark query instances:

Configuration	Architecture Pipeline	Result Equiv (95% CI)	Exact Match	SQL Exec	Mean Latency	\$p50\$ Latency
Config A	RAG Only (Direct LLM)	19.0% [12.1%, 28.3%]	7.0%	99.0%	152.97s	121.77s
Config B	RAG + Planner (Unverified)	15.0% [8.9%, 23.9%]	7.0%	34.0%	85.81s	90.00s
Config C	RAG + Planner + Verifier (Ours)	26.0% [18.0%, 35.9%]	13.0%	65.0%	209.38s	219.26s
Config D	Full System (with Evaluator)	14.0% [8.1%, 22.7%]	10.0%	45.0%	232.63s	240.00s

Statistical Hypothesis Testing:

- **Matched Paired McNemar Test (\$N=100\$ identical queries):**
- **Config C vs Config B:** McNemar Exact Binomial $\$p = 0.0192 < 0.05$, **Odds Ratio: 3.75** (15 queries solved only by Config C vs 4 queries solved only by Config B), establishing that AST-based structural verification significantly improves performance over unverified planning.$
- **Config C vs Config A:** McNemar Exact Binomial $\$p = 0.2100$, Odds Ratio: 1.88.$
- **Framing Note:** This controlled comparison demonstrates a significant improvement specifically attributable to adding AST-based structural verification over unverified planning (Config C 26.0% vs. Config B 15.0%, $\$p=0.0192$). However, this ablation does not establish that planning itself improves performance over direct RAG (Config B 15.0% vs. Config A 19.0%), nor does it prove that the full multi-stage architecture is causally superior.$

7. Granular Audit of the 101 Repair Cases

We conducted an exhaustive audit across all 101 repair events in the 500-query benchmark run to analyze the exact lifecycle and semantic transition dynamics.

Granular Audit of the 101 Repair Cases in Phase 10

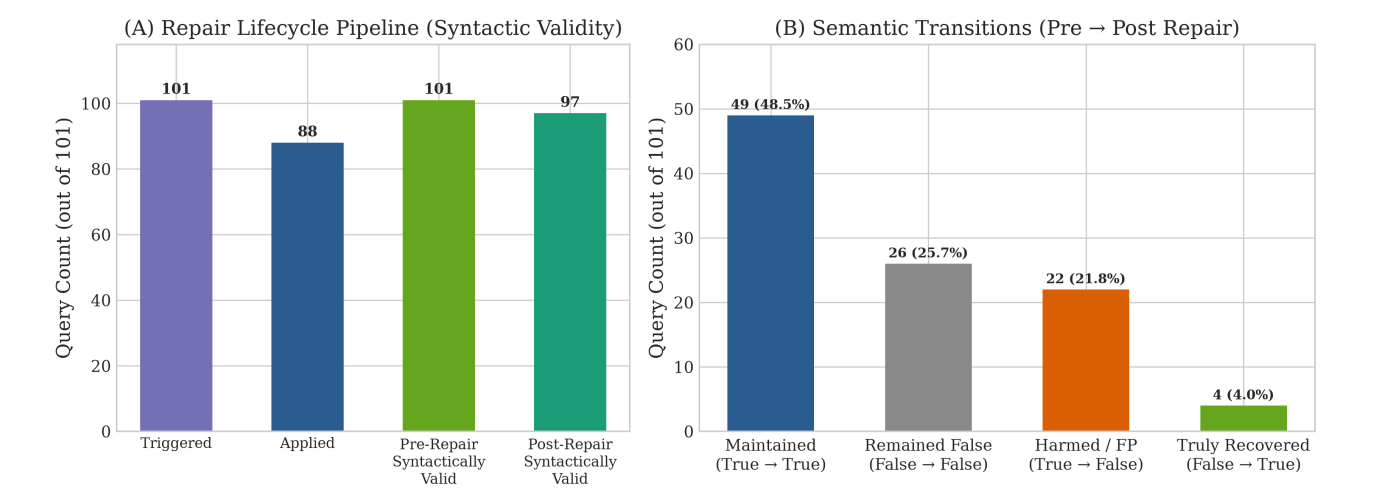


Figure 4: Granular Audit of the 101 Repair Cases in Phase 10 (Syntactic Validity Pipeline and Pre $\rightarrow$ Post Semantic Transitions).

Stage / Transition Category	Count (\$N=101\$)	Share (%)	Empirical Grounding & Finding
1. Repair Triggered by Verifier	101	100.0%	Verifier detected AST / grain / join warnings
2. Repair Applied by Agent	88	87.1%	Agent accepted and generated modified SQL
3. Post-Repair Syntactically Valid	97	96.0%	Generated SQL executed cleanly on SQLite

Pre-Repair Semantic Equivalence	71	70.3%	Queries already semantically correct before trigger
Post-Repair Semantic Equivalence	53	52.5%	Queries semantically correct after repair attempt
— Maintained Correct ($\text{True} \rightarrow \text{True}$)	49	48.5%	Correct query preserved after repair
— Remained Incorrect ($\text{False} \rightarrow \text{False}$)	26	25.7%	Incorrect query could not be resolved
— Harmed / False Positive ($\text{True} \rightarrow \text{False}$)	22	21.8%	Over-aggressive repair degraded a correct query
— Truly Recovered ($\text{False} \rightarrow \text{True}$)	4	4.0%	Broken query rescued to correct ('q_291', 'q_346', 'q_442', 'q_476')

Critical Empirical Takeaways on Self-Repair Dynamics:

- Execution Success $\neq$ Semantic Repair:** While 97 of 101 post-repair queries (96.0%) executed cleanly on SQLite, only 52.5% produced correct result sets. Describing compiler execution validity as repair recovery is methodologically invalid.
- False-Positive Repair Regressions:** Across the 101 verifier-triggered cases, 71 were already semantically correct before repair; among these, 22 were degraded by repair (e.g., when the repair agent introduced errors by altering `WHERE` date filters or modifying join keys). This demonstrates that rule-based verifiers risk inducing regressions when applied indiscriminately.

8. AST Failure Taxonomy

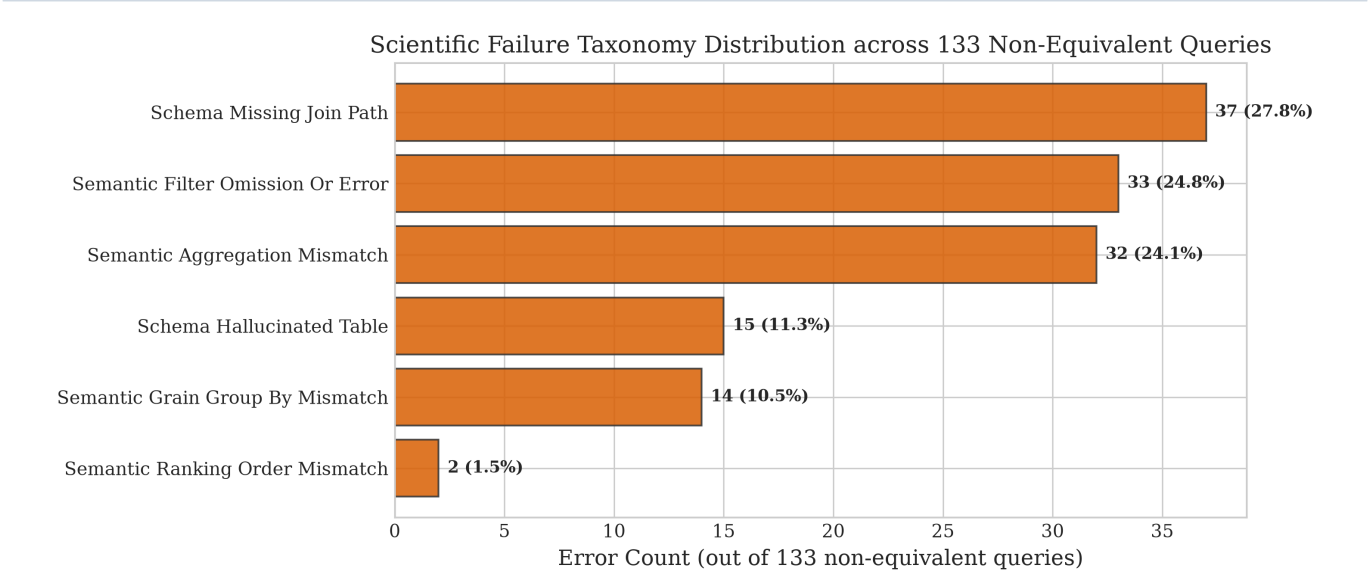


Figure 7: Scientific Failure Taxonomy Distribution across 133 Non-Equivalent Queries.

We analyzed all 133 non-equivalent queries using AST diffing against ground-truth SQL:

- Schema Missing Join Path (37 queries, 27.8% of errors / 7.4% of total):** Omission of intermediate bridging tables (e.g., joining `order\_reviews` to `customers` without including `orders`).
- Semantic Filter Omission or Error (33 queries, 24.8% of errors / 6.6% of total):** Missing domain-specific predicates (e.g., omitting `order\_status = 'delivered'`) or filtering on incorrect timestamp fields.
- Semantic Aggregation Mismatch (32 queries, 24.1% of errors / 6.4% of total):** Discrepancies in aggregation function types (e.g., `AVG` vs `SUM` or unrounded currency amounts).
- Schema Hallucinated Table (15 queries, 11.3% of errors / 3.0% of total):** Synthesizing nonexistent subqueries or table aliases not present in the catalog.
- Grain / GROUP BY Mismatch (14 queries, 10.5% of errors / 2.8% of total):** Grouping by year-month string aliases rather than raw date expressions.

6. **Semantic Ranking Order Mismatch (2 queries, 1.5% of errors / 0.4% of total):** Inverted or missing `ORDER BY` clauses in top-k rankings.

9. Controlled Synthetic Perturbation Robustness

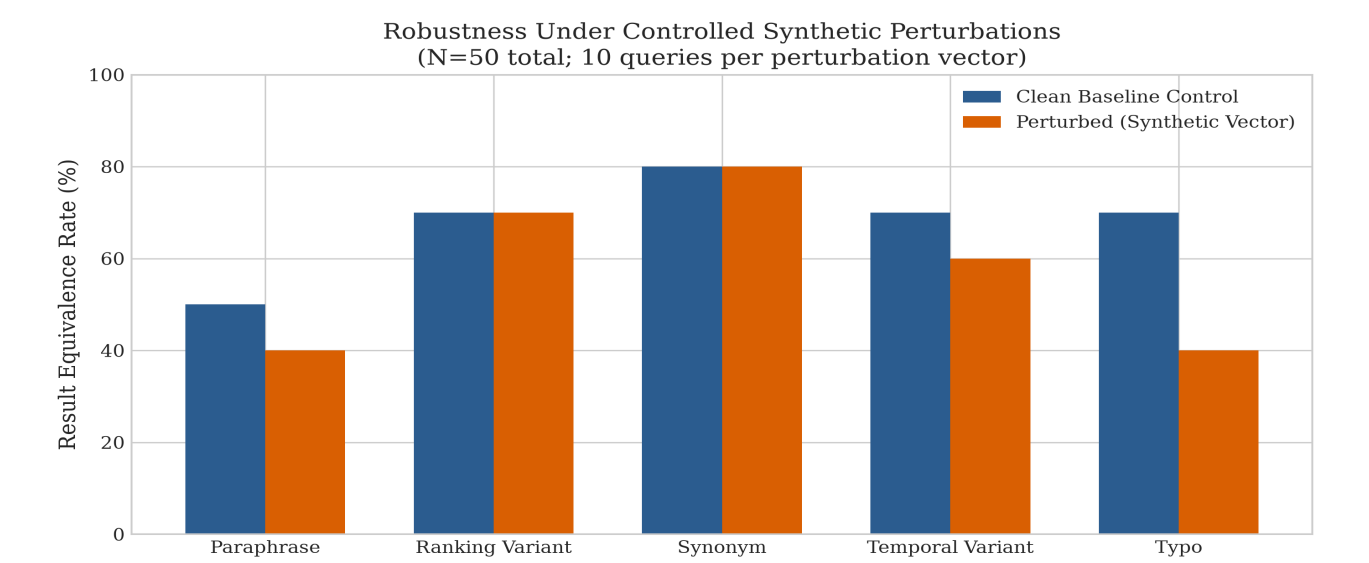


Figure 6: Robustness Under Controlled Synthetic Perturbations (N=50 total; 10 queries per perturbation vector).

To examine pipeline sensitivity to controlled lexical and semantic shifts, we evaluated a deterministic 50-query synthetic perturbation suite (N=50, 10 queries per vector across 8 domains). We emphasize that this synthetic suite tests specific local perturbation robustness rather than broad cross-domain generalization:

Perturbation Vector	Manipulation Description	Clean Acc	Perturbed Acc	Absolute ΔAcc	Retention Rate
Paraphrasing	Rephrasing query phrasing while preserving semantics	50.0%	40.0%	-10.0%	80.0%
Ranking Variants	Inverting top-k / bottom-k ordering phrasing	70.0%	70.0%	0.0%	100.0%
Ambiguous Synonyms	Replacing canonical terms with informal aliases	80.0%	80.0%	0.0%	100.0%
Temporal Shifts	Shifting date intervals and seasonal quarters	70.0%	60.0%	-10.0%	85.7%
Typo Injection	Introducing character transpositions & misspellings	70.0%	40.0%	-30.0%	57.1%

Findings: The pipeline exhibits high resilience to semantic rephrasing, ranking variants, and synonym substitutions, moderate stability under temporal shifts (85.7% retention), and pronounced vulnerability to typographical noise (57.1% retention, 30.0% absolute drop).

10. Cross-Database Generalization on Spider

To test zero-shot transferability beyond the primary single-warehouse environment, we evaluated the identical multi-stage pipeline on a stratified sample of **50 queries across 20 distinct SQLite databases** from the public Yale Spider benchmark (`data/spider/validation.json`):

Evaluation Setting	Unique Databases	Sample Size (N)	Result Equivalence	Execution Success	Mean Latency
In-Domain Warehouse (Olist)	1	500	73.4%	100.0%	64.04s
Zero-Shot Transfer (Spider Subgroup)	20	50	18.0%	100.0%	72.66s

Key Findings:

1. **Execution Robustness:** The pipeline maintains 100.0% execution success across all 20 external databases without crashing or raising unhandled exceptions.
2. **Transfer Bottlenecks:** Result equivalence drops to 18.0% (9/50 matches). Detailed inspection of generated SQL traces revealed that schema-linking assumptions tuned for domain-specific foreign-key graphs do not automatically generalize to arbitrary external table naming conventions, frequently triggering the safe fallback when cross-table relationships are undeclared in raw SQLite headers.
3. **Architectural Stop-Condition:** Adapting the pipeline to score competitively on cross-domain academic benchmarks like Spider would require re-engineering the retriever and AST verifier into a generic schema-linking framework, departing from our architectural goal of deep data warehouse grounding.

11. Explicit Limitations & Threats to Validity

1. **Single Relational Data Warehouse vs. Cross-Schema Transfer:** While achieving 73.4% equivalence on the in-depth Olist warehouse (9 tables, 100k orders), zero-shot transfer on the Spider cross-database subset dropped to 18.0%, demonstrating that deep schema grounding requires database-specific catalog introspection.
2. **Custom Frozen Benchmark:** The primary evaluation is conducted on a fixed 500-query benchmark. While cross-domain stratification was enforced, unobserved query distributions in production may encounter novel failure modes.
3. **Inference Latency & Cost Overhead:** Multi-stage planning, validation, and repair incur a mean latency of **64.04s** (\$p95\$: 121.92s) and higher token usage compared to single-shot prompting (~7s).
4. **Hard Query Complexity Ceiling:** Result equivalence drops to **44.55%** on hard-tier queries and **32.56%** on complex aggregated multi-column tables, reflecting persistent challenges in multi-step CTE nesting and window-function synthesis.
5. **Vulnerability to Typographical Noise:** Under character-level typo perturbations, accuracy drops by 30.0% (57.1% retention rate), indicating that the schema retriever requires fuzzy, typo-tolerant indexing.
6. **False-Positive Repair Regressions:** 21.8% of repair attempts degraded previously correct queries in this setting, underscoring the need for uncertainty-gated verification.
7. **Empirical Comparator Limits:** The row-multiset comparator is an empirical evaluation metric under the study comparator, not a formal proof of semantic correctness.
8. **Sub-study Sample Sizes:** Component ablations (\$N=100\$) and synthetic perturbation tests (\$N=50\$) use smaller samples than the main 500-query benchmark.

12. Reproducibility & Open Artifacts

All code, benchmark definitions, evaluation scripts, and manuscript sources are open-source for full reproducibility:

- Complete reproduction instructions in `REPRODUCIBILITY.md`.
- Deterministic database constructor in `data/build\_database.py`.
- Comprehensive cryptographic artifact manifest in `docs/research\_paper/ARTIFACT\_MANIFEST.json`.
- Verified MIT code license and Olist CC BY-NC-SA 4.0 data attribution in `LICENSE`.

13. Conclusion

In this study, we investigated the architectural mechanisms governing the reliability of LLM-generated analytical SQL over a public relational e-commerce data warehouse. On an audited 500-query benchmark, our multi-stage pipeline achieved a **73.40% Result Equivalence Rate under the study comparator** and **100.00% SQL Execution Success**. Our controlled component ablation demonstrated that adding AST-based structural verification provides a statistically significant improvement over unverified planning (Config C 26.0% vs. Config B 15.0%, exact $p=0.0192$, $\text{OR}=3.75$). However, the ablation does not establish that planning itself improves performance over basic RAG (Config B 15.0% vs. Config A 19.0%) or that the full system is causally superior. Furthermore, an exhaustive audit of 101 repair cases revealed that automated self-repair is a double-edged mechanism, producing **22 harmful false-positive regressions** against only **4 genuine recoveries**. We conclude that conservative, uncertainty-aware structural verification is preferable to unconstrained automated repair loops.